UNIT-II

Graphs & Divide and Conquer

Syllabus:

Trees: Heap Trees (Priority Queues) – Min and Max Heaps, Operations and Applications.

Graphs: Terminology, Representations, Graph Traversals, Minimum spanning trees – prims algorithm, krushkal algorithm.

Divide and Conquer: General Method, Defective chessboard, finding the Maximum and Minimum numbers, Quick sort.

HEAP:

A Heap tree is a complete binary tree with the property of the value at each node is as larger as (as smaller as) the value of its child nodes. Heap tree is also called Binary Heap.

There are two types of heaps, and they are as follows...

1. Max Heap (Root node must be greater than its child nodes)
2. Min Heap (Root node must be lesser than its child nodes)

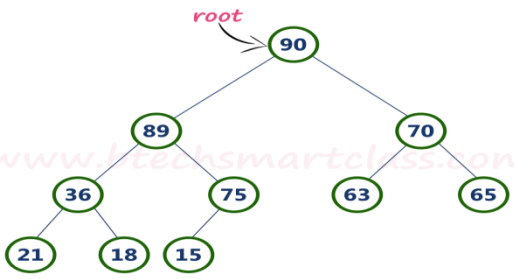
Every heap data structure has the following properties...

Property #1 (Ordering): Nodes must be arranged in an order according to values based on Max heap or Min heap.

Property #2 (Structural): All levels in a heap must be full, except last level and nodes must be filled from left to right strictly.

Max Heap/ Max Tree:

- Max heap is a specialized complete binary tree. It is also called max tree. A max tree is a tree in which the value of parent node is larger than the values of its children.
- The following tree is satisfying both Ordering property and Structural property according to the Max Heap data structure.



Operations on Max Heap:

The following operations are performed on a Max heap data structure...

1. **Finding Maximum**
2. **Insertion**
3. **Deletion**

Finding Maximum Value Operation in Max Heap

Finding the node which has maximum value in a max heap is very simple. In a max heap, the root node has the maximum value than all other nodes. So, directly we can display root node value as the maximum value in max heap.

Insertion Operation in Max Heap:

Algorithm

Step 1: Insert the newNode as last leaf from left to right based on structured property.

Step 2: Compare newNode value with its Parent node.

Step 3: If newNode value is greater than its parent, then swap both.

Step 4: Repeat step 2 and step 3 until newNode value is less than its parent node (or) newNode reached to root.

Example:

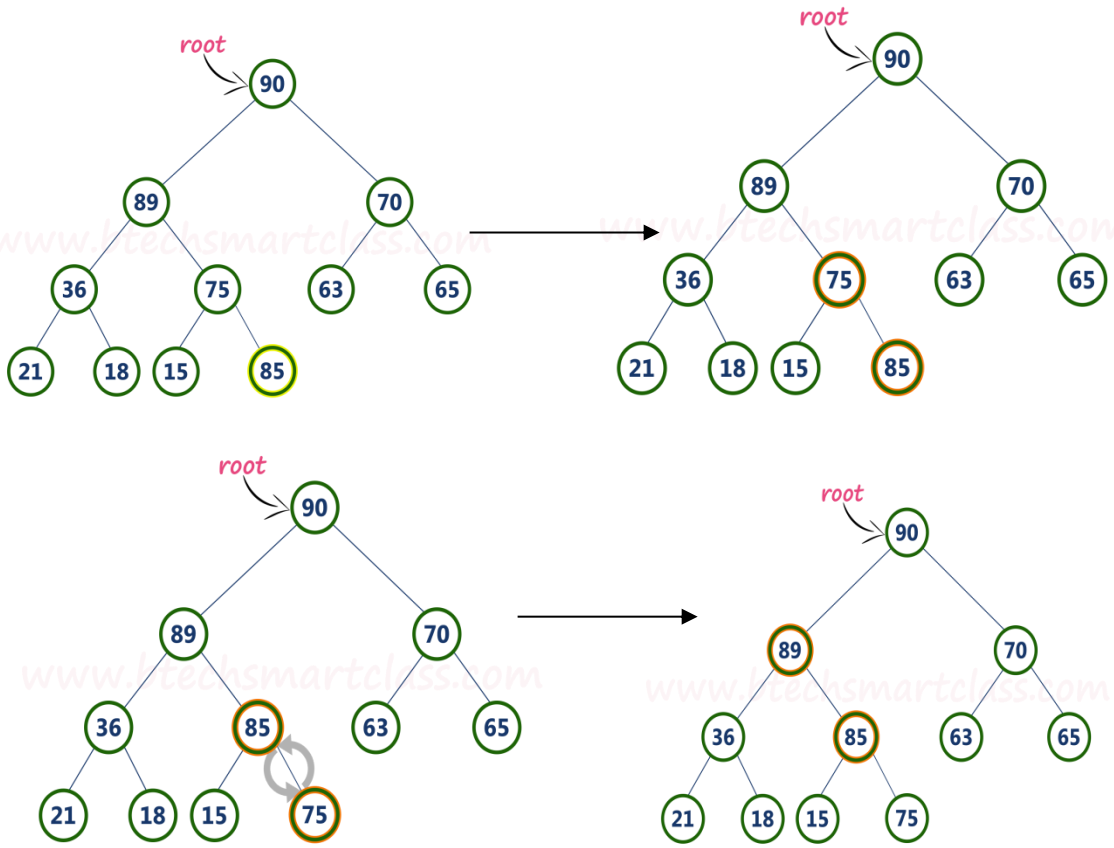
Consider the above max heap. **Insert a new node with value 85.**

Step 1 - Insert the **newNode** with value 85 as **last leaf** from left to right. That means newNode is added as a right child of node with value 75.

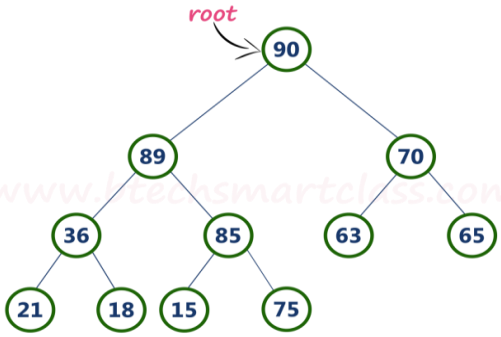
Step 2 - Compare **newNode value (85)** with its **Parent node value (75)**. That means **85 > 75**

Step 3 - Here **newNode value (85) is greater** than its **parent value (75)**, then **swap** both. After swapping, max heap is...

Step 4 - Now, again compare newNode value (85) with its parent node value (89).



Here, newNode value (85) is smaller than its parent node value (89). So, we stop insertion process. Finally, max heap after insertion of a new node with value 85 is as follows...



Deletion Operation in Max Heap

Deleting root node from a max heap is little difficult as it disturbs the max heap properties. We use the following steps to delete the root node from a max heap...

Algorithm:

Step 1 − Remove root node.
Step 2 − Move the last level last element from right side to root.
Step 3 − Heapify (Fix the heap):
if the heap property holds true
then you are done.
else if the rootnode_value <= its child nodes value
swap the rootnode with the largest child node and repeat step 3.

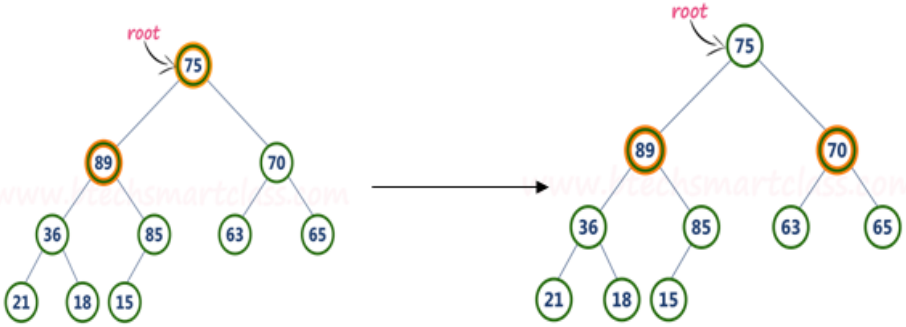
Example

Consider the above max heap. **Delete root node (90) from the max heap.**

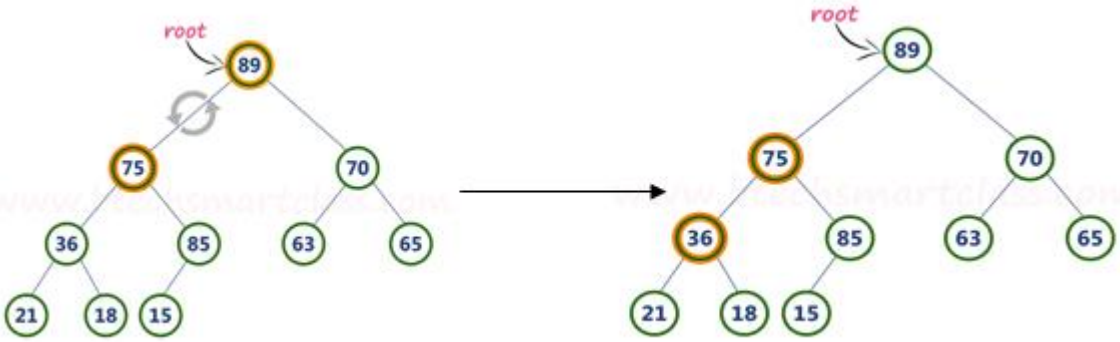
Step 1 - Swap the **root node (90)** with **last node 75** in max heap. After swapping max heap is



- **Step 2 - Delete** last node. Here the last node is 90. After deleting node with value 90 from heap, max heap is as follows...
- **Step 3 -** Compare **root node (75)** with its **left child (89)**. Here, **root value (75) is smaller** than its left child value (89). So, compare left child (89) with its right sibling (70).



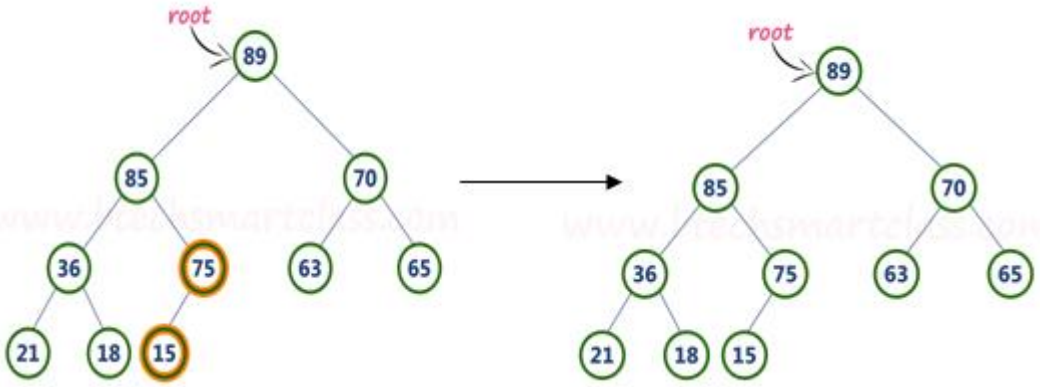
- **Step 4 -** Here, **left child value (89) is larger** than its **right sibling (70)**, So, **swap root (75)** with **left child (89)**.



- **Step 5 -** Now, again compare **75** with its **left child (36)**. Here, node with value **75** is larger than its left child. So, we compare node **75** with its right child **85**.
- **Step 6 -** Here, node with value **75** is smaller than its **right child (85)**. So, we swap both of them. After swapping max heap is as follows...



- **Step 7 -** Now, compare node with value **75** with its left child (**15**). Here, node with value **75** is larger than its left child (**15**) and it does not have right child. So we stop the process. Finally, max heap after deleting root node (**90**) is as follows...

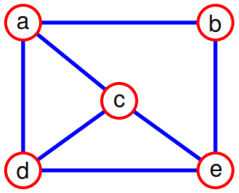


Applications:

1. Priority Queues: Heaps are commonly used to implement priority queues, where elements with higher priority are extracted first.
2. Sorting Algorithms: Heapsort, a comparison-based sorting algorithm, is implemented using the Heap data structure. It has a time complexity of O(n log n), making it efficient for large datasets.

3. Graph algorithms: Heaps are used in graph algorithms such as Prim's Algorithm, Dijkstra's algorithm., and the A* search algorithm.
4. Medical Applications: In medical applications, heaps are used to store and manage patient information based on priority, such as vital signs, treatments, and test results.
5. Resource allocation: Heaps can be used to efficiently allocate resources in a system, such as memory blocks or CPU time, by assigning a priority to each resource and processing requests in order of priority.

Graph Terminology:



A graph is defined as **G = (V, E):**

V: set of vertices

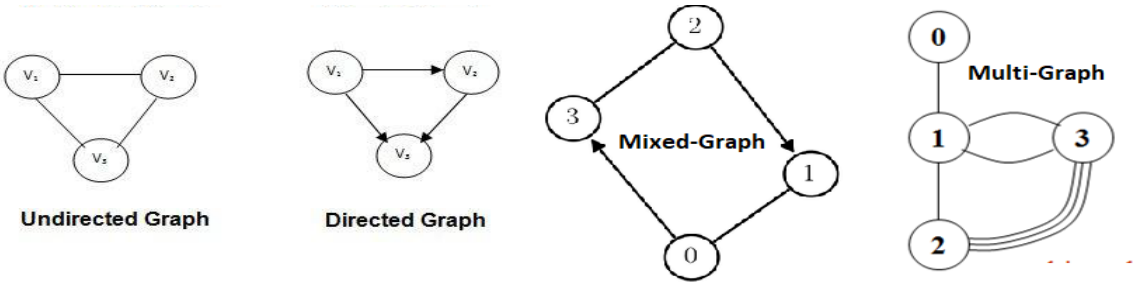
E: set of edges connecting the vertices in V,

An edge E = (u,v) is a pair of vertices.

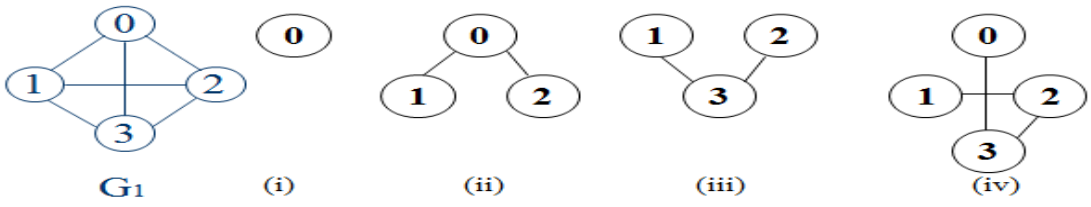
V= {a,b,c,d,e}

E={(a,b),(a,c),(a,d),(b,e),(c,d),(c,e),(d,e)}

- **Undirected Graph**: An Undirected graph is a graph, in which all the edges have no direction. The edge (u,v) is identical to (v,u).
- **Directed Graph:** A directed graph (di-graph) is a graph, in which all the edges have direction.
- **Mixed Graph:** A mixed graph is a graph, in which some of the edges having the direction and some of the edge are not having the direction.
- **Multi-Graph:** In a multi-graph, there can be more than one edge from vertex P to vertex Q. In a simple graph there is at most one.



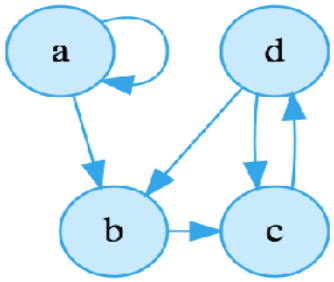
- **Sub-Graph:** A sub-graph of G is G', it consists of V(G') is a subset of V(G) and E(G') is a subset of E(G)



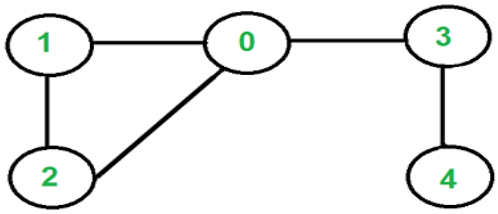
Some of the subgraph of G_1

- **Adjacency**: Let an edge E is having in between pair of vertices (u,v) then u and v are adjacent to each other.
  - (v0, v1) is an edge in an undirected graph, v0 and v1 are adjacent
  - (v0, v1) is an edge in an directed graph, v0 is adjacent to v1, and v1 is adjacent from v0

- **Path:** It is sequence of vertices; every vertex is adjacent to the next vertex.
- **Cycle:** It is path containing minimum of three vertices, the stating vertex and last vertex must be same.
- **Loop (Self-Edge):** A self loop is an edge that connects a vertex to itself.



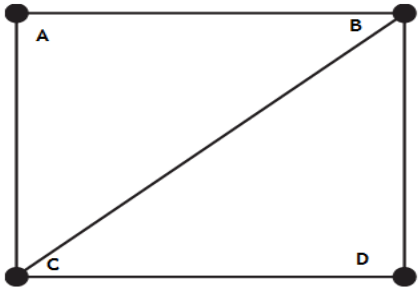
Self Loop at Vertex a



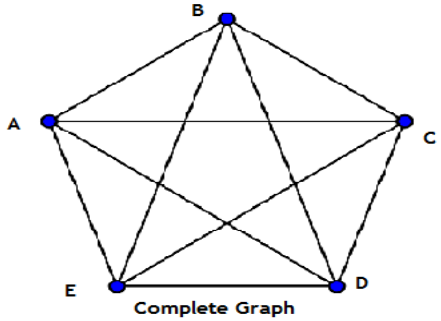
cycle 1-2-0-1
cycle 0-1-2-0
cycle 2-0-1-2

path 2-0-3-4
path 2-1-0-3-4

- **Connected Graph**: A graph is said to be connected, if there exists a path between any two vertices u and v otherwise the graph is disjoint or disconnected graph.
  - The total number of edges in connected graph is **n-1**
- **Complete Graph**: A graph is said to complete graph if there exist an edge from every vertex to every other vertex.
  - The total number of edges in undirected complete graph is **n(n-1)/2**
  - The total number of edges in directed complete graph is **n(n-1)**



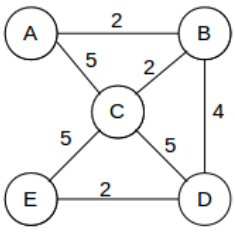
Connected Graph



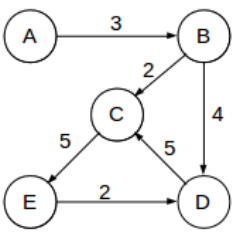
Complete Graph

- **Weighted Graph**: A graph is said to be a weighted graph if every edge in the graph is assigned by some weights or values or lengths.
- **Degree:** The degree of a vertex is the number of edges incident to that vertex

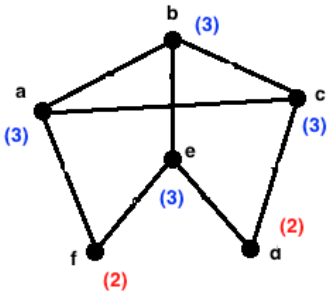
  For directed graph,
  - The in-degree of a vertex v is the numbers of edges are terminated (head).
  - The out-degree of a vertex v is the numbers of edges are started (tail)



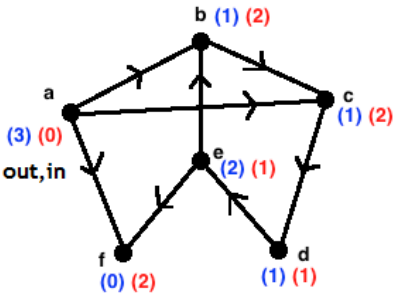
Undirected



Directed



Un-Directed Graph



Directed Graph

Graph Representations:

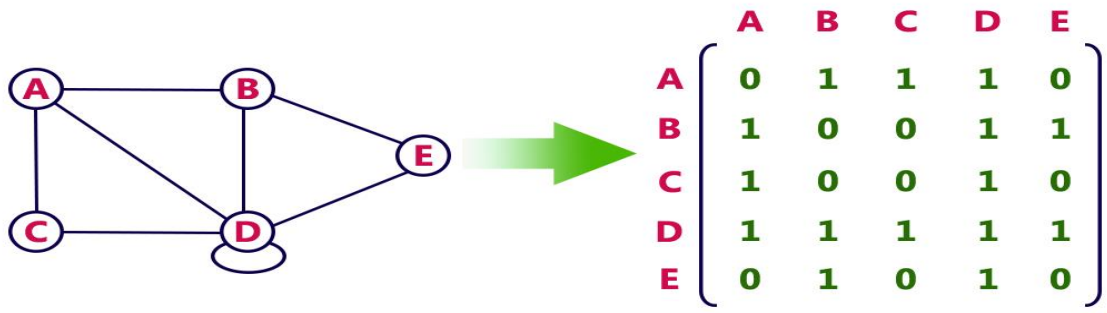
Graph data structure is represented using the following representations...

1. Adjacency Matrix (sequential or Array Representation)
2. Adjacency List (Linked List Representation)

1. Adjacency Matrix:

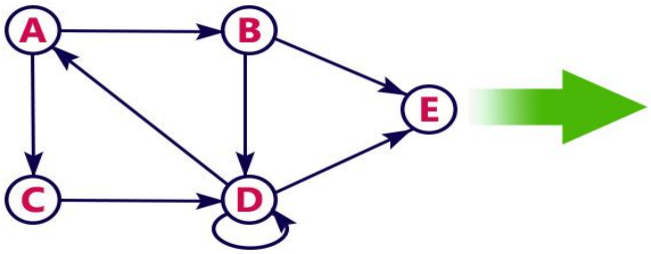
In this representation, the graph is represented using a matrix of size total number of vertices by a total number of vertices. That means a graph with 4 vertices is represented using a matrix of size 4X4. In this matrix, both rows and columns represent vertices. The elements are filled with either 1 or 0. Here, 1 represents that there is an edge from row vertex to column vertex and 0 represents that there is no edge from row vertex to column vertex.

For example, consider the following undirected graph representation...



	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	1	1
C	1	0	0	1	0
D	1	1	1	1	1
E	0	1	0	1	0

Directed graph representation...



	A	B	C	D	E
A	0	1	1	0	0
B	0	0	0	1	1
C	0	0	0	1	0
D	1	0	0	1	1
E	0	0	0	0	0

- The sum of all values in a row is equivalent to the out degree of a vertex.
- The sum of all values in a column is equivalent to the in degree of a vertex.

Advantages:

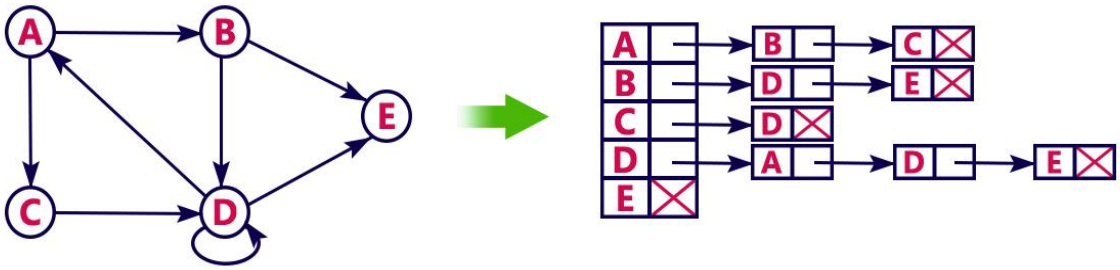
- Easy to store and manipulate matrices.
- Finding the cycles and path in a graph is easy.

Disadvantage: The space requires for a graph having N vertices is NXN, but the matrix contain only few elements.

2. Adjacency List:

- In this representation, every vertex of a graph contains list of its adjacent vertices.
- The structure contains list of all the vertices, every vertex having its own link to its own list, the list contain all adjacent vertices.

For example, consider the following graph representation implemented using linked list...



Advantage: It clearly shows the adjacent vertices for a vertex are.

Graph Traversals:

Traversals mean visiting all the vertices in a graph at exactly once in a systematic way.

There are two types of graph traversals, they are.

1. Depth First Search (DFS)
2. Breadth First Search (BFS)

Depth First Search (DFS):-

- In DFS we are using the application STACK.
- In graphs there is no root node or start vertex, so we can take any arbitrary vertex as starting vertex.

The process/Algorithm of DFS is

i. Push the start vertex in to STACK

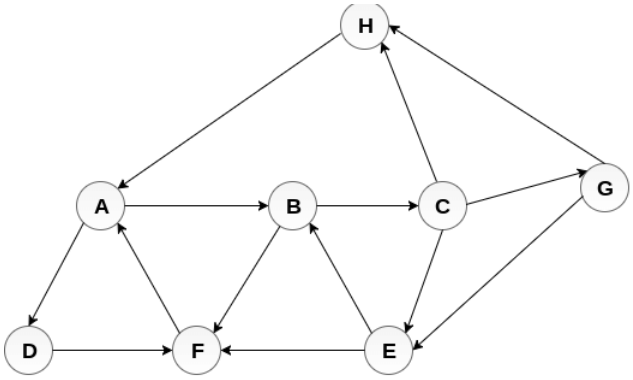
ii. Repeat process until STACK is empty

   a. Pop the vertex at top of the STACK

   b. Process the vertex

   c. Push the adjacent vertices in to the STACK for popped vertex.

Example: Consider the graph G along with its adjacency list, given in the figure below.



Adjacency Lists

A : B, D
B : C, F
C : E, G, H
G : E, H
E : B, F
F : A
D : F
H : A

Solution:

Let the starting vertex of the graph is H.

1. Push H onto the stack

   STACK : H

2. POP the top element of the stack i.e. H, print it and push all the adjacent vertices of H onto the stack that are is ready state.

   Print H

   STACK : A
3. Pop the top element of the stack i.e. A, print it and push all the adjacent vertices of A onto the stack that are in ready state.

   Print A

   Stack : B, D
4. Pop the top element of the stack i.e. D, print it and push all the adjacent vertices of D onto the stack that are in ready state.

   Print D

   Stack : B, F
5. Pop the top element of the stack i.e. F, print it and push all the adjacent vertices of F onto the stack that are in ready state.

   Print F

   Stack : B
6. Pop the top of the stack i.e. B and push all the adjacent vertices

   Print B

   Stack : C
7. Pop the top of the stack i.e. C and push all the adjacent vertices.

   Print C

   Stack : E, G
8. Pop the top of the stack i.e. G and push all its adjacent vertices.

   Print G

   Stack : E
9. Pop the top of the stack i.e. E and push all its adjacent vertices.

   Print E

   Stack :

Hence, the stack now becomes empty and all the nodes of the graph have been traversed.

The printing sequence of the graph using DFS is: H → A → D → F → B → C → G → E

Breadth First Search (BFS):-

- In BFS we are using the application of QUEUE.
- In graphs there is no root node or start vertex, so we can take any arbitrary vertex as starting vertex.

The process/Algorithm of BFS is

i. Enqueue the start vertex in to QUEUE

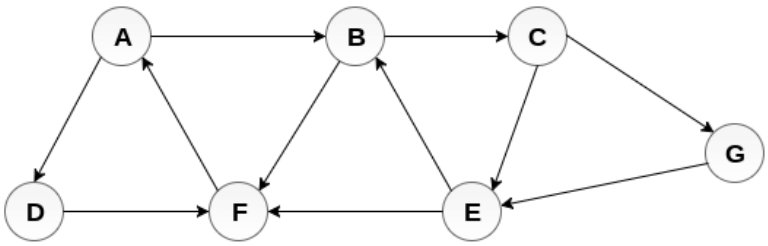
ii. Repeat process until QUEUE is empty

a. Dequeue the vertex at front of the QUEUE

b. Process the vertex

c. Enqueue the adjacent vertices in to the QUEUE at rear end, for dequeue vertex.

Example: Consider the graph G along with its adjacency list, given in the figure below.



Adjacency Lists

A : B, D
B : C, F
C : E, G
G : E
E : B, F
F : A
D : F

Solution: Lets start vertex of the graph is Node A.

1. Insert node A into QUEUE

QUEUE = {A}

2. Delete the Node A from QUEUE and insert all its adjacent vertices.

QUEUE = {B, D}

Print A

3. Delete the node B from QUEUE and insert all its adjacent vertices.

QUEUE = {D, C, F}

Print B

4. Delete the node D from QUEUE and insert all its adjacent vertices. Since F is the only neighbour of it which has been inserted, we will not insert it again.

QUEUE = {C, F}

Print D

5. Delete the node C from QUEUE and insert all its adjacent vertices.

QUEUE = {F, E, G}

Print C

6. Remove F from QUEUE and add all its adjacent vertices. Since all of its adjacent vertices has already been added, we will not add them again.

QUEUE = {E, G}

Print F

7. Remove E from QUEUE, all of E's adjacent vertices has already been added to QUEUE therefore we will not add them again.

QUEUE = {G}

Print E

8. Remove G from QUEUE, the adjacent vertices has already been added to QUEUE therefore we will not add them again.

QUEUE = { }

Print G

Hence, the QUEUE is now becomes empty and all the nodes of the graph have been traversed.

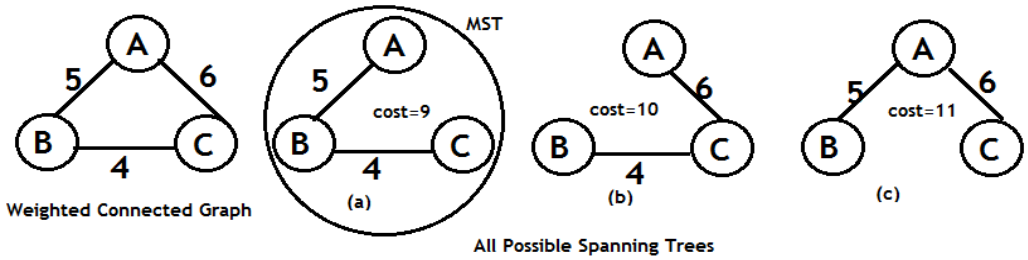
The printing sequence of the graph using BFS is: A → B → D → C → F → E → G

Spanning Tree: -

- A spanning tree of a graph is a subgraph in includes all of the vertices of a graph, without any cycles.
- A graph may have one or more number of possible spanning trees.
- If a graph contains N vertices then the total numbers of possible spanning trees are $\mathbf{N^{N-2}}$ and every spanning tree is having maximum number of edges will be **N-1.**

Minimum cost Spanning Trees:

- In the weighted connected graph, **the minimum cost-spanning tree** is a spanning tree that has minimum weights than all other spanning trees.
- Weight or cost of spanning tree is sum of weights of all edges in the spanning tree.



There are two popular techniques for finding the minimum cost-spanning tree for any graph.

1. Kruskal's Algorithm
2. Prim's Algorithm

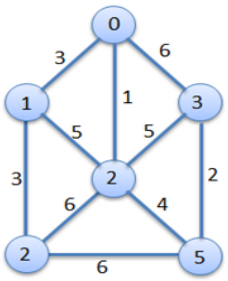
1. Kruskal's Algorithm:

- Every time picking the minimum weighted edge from the graph and draw the edge between vertices in minimum cost spanning tree and it shouldn't form any cycles.
- The same process is repeated until all the vertices are connected then we will find the minimum cost spanning tree for the given graph.

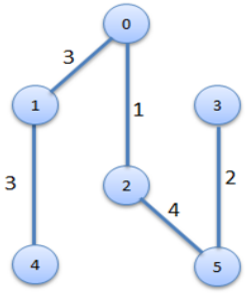
Procedure:

1. Sort all Edges of weights in ascending order
2. Initially the MST is empty
3. Repeat until the graph containing N-1 number of edges
   a. Pick an edge and insert into MST, if it is not forming the cycle then consider the edge.
   b. Otherwise remove the edge from the MST.
4. Return MST.

Example:



A Simple Weighted Graph



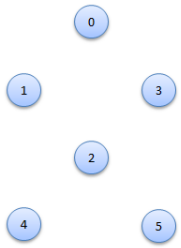
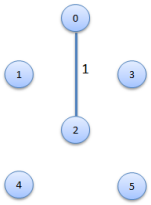
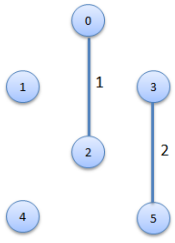
Minimum-Cost Spanning Tree

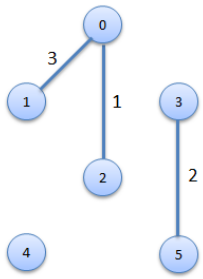
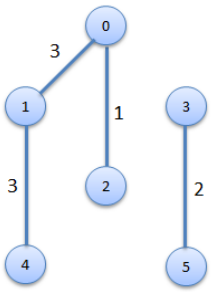
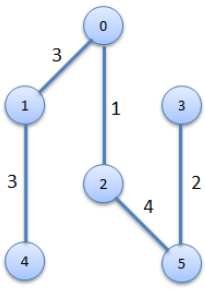
Procedure for finding Minimum Spanning Tree

Step1. Edges are sorted in ascending order by weight.

Edge No.	Vertex Pair	Edge Weight
E1	(0,2)	1
E2	(3,5)	2
E3	(0,1)	3
E4	(1,4)	3
E5	(2,5)	4
E6	(1,2)	5
E7	(2,3)	5
E8	(0,3)	6
E9	(2,4)	6
E10	(4,5)	6

Step2. Edges are added in sequence.

Initial Empty MST	
Add Edge E1	
Add Edge E2	

Add Edge E3	
Add Edge E4	
Add Edge E5	

Minimum Cost = 1+2+3+3+4 = 13

Algorithm:

```
Algorithm Kruskals(G)
{
   // Input: Graph G, Output: ET (Tree with minimum edges)
   Sort E;
   ET := ϕ;
   edge_counter := 0;
   while (edge_counter < |V|-1) do
   {
      if (ET U {ei} is acyclic) then
        {
         ET <-- ET U {ei};
         edge_counter := edge_counter + 1
        }
   }
   return ET ;
}
```

The time complexity of Kruskal's algorithm is ***O(E log V) or O(E log E).***

2. Prim's Algorithm:

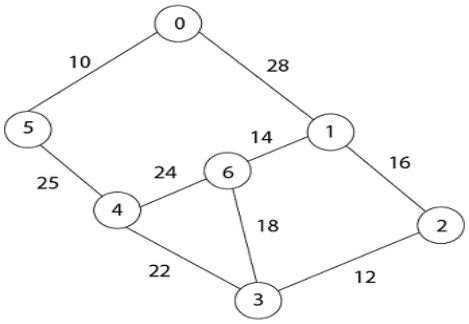
- Prim's Algorithm used to find the minimum cost-spanning tree.
- It follows greedy algorithm. It starts with an empty spanning tree MST. At every step, it considers all the edges and picks the minimum weight edge. After picking the edge, it moves the other endpoint of edge to set containing MST.

Procedure:

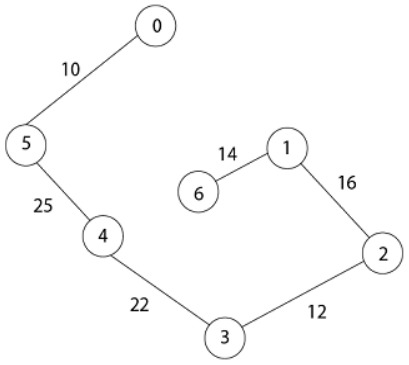
1. Create MST set that keeps track of vertices and included in MST.

2. Assign key values to all vertices in the input graph. Initialize all key values as INFINITE (∞). Assign key values like 0 for the start vertex so that it is picked first.
3. While MST set doesn't include all vertices.
    a. Pick vertex **u** which is not is MST set and has minimum key value. Include 'u' to MST set.
    b. Update the key value of all adjacent vertices of **u**. To update, iterate through all adjacent vertices. For every adjacent vertex **v**, if the weight of edge (u,v) less than the previous key value then update key value as a weight of new edge.

Example:



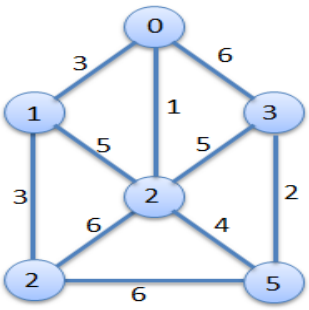
Weighted Connected Graph



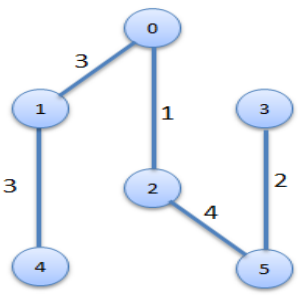
Minimum Cost Spanning Tree

VERTICES	0	1	2	3	4	5	6		
Key-Value	**0**	∞	∞	∞	∞	∞	∞	MST=[0]	
Key-Value, Adj(0)	-	28(0)	∞	∞	∞	**10(0)**	∞	MST=[0,5]	edge(0,5)=10
Key-Value, Adj(5)	-	28(0)	∞	∞	**25(5)**	-	∞	MST=[0,4,5]	edge(5,4)=25
Key-Value, Adj(4)	-	28(0)	∞	**22(4)**	-	-	24(4)	MST=[0,3,4,5]	edge(4,3)=22
Key-Value, Adj(3)	-	28(0)	**12(3)**	-	-	-	18(3)	MST=[0,2,3,4,5]	edge(3,2)=12
Key-Value, Adj(2)	-	**16(2)**	-	-	-	-	18(3)	MST=[0,1,2,3,4,5]	edge(2,1)=16
Key-Value, Adj(1)	-	-	-	-	-	-	**14(1)**	MST=[0,1,2,3,4,5,6]	edge(1,6)=14
Key-Value, Adj(6)			*All the vertices in the MST*					Total Cost = 10+25+22+12+16+14 = 99	

Example:



A Simple Weighted Graph

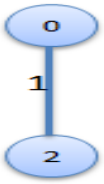


Minimum-Cost Spanning Tree

Procedure for finding Minimum Spanning Tree

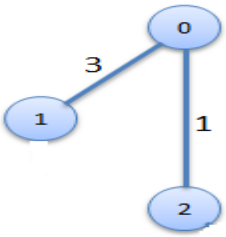
Step1

No. of Nodes	0	1	2	3	4	5
Distance	-	3	**1**	6	∞	∞
Distance From		0	0	0		



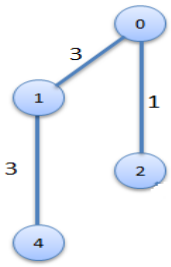
Step2

No. of Nodes	0	1	2	3	4	5
Distance	-	**3**	-	5	6	4
Distance From		0		2	2	2



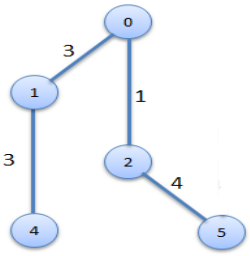
Step3

No. of Nodes	0	1	2	3	4	5
Distance	-	-	-	5	**3**	4
Distance From				2	1	2



Step4

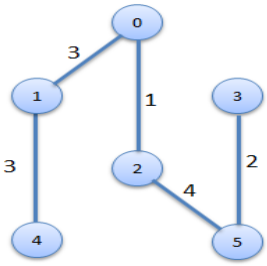
No. of Nodes	0	1	2	3	4	5
Distance	-	-	-	5	-	**4**
Distance From				2		2



Step5

No. of Nodes	0	1	2	3	4	5
Distance	-	-	-	**3**	-	-
Distance From				2		

Minimum Cost = 1+2+3+3+4 = 13



Algorithm:

```
Algorithm Prims(G)
{
   // Input: Graph G, Output: ET (Tree with minimum edges)
   initialize(Q);
   for every vertex v in V
      dv := ∞;
   ds :=0;
   VT := ϕ;
   for i := 0 to |V|-1 do
   {
      u* := delete_min(Q);
      VT := VT U {u*};
      for every vertex u in V-VT adj to u* do
      {
         if (w(u*,u) < du) then
            du := w(u*,u);
      }
   }
}
```

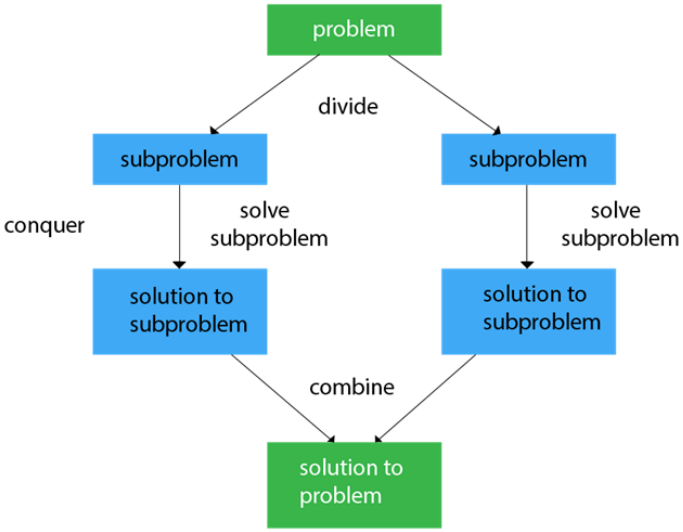
The time complexity of Prim's algorithm is **O(V^2).**

Divide-and-Conquer Method:

- Divide-and-conquer are probably the best-known general algorithm design techniques. The principle behind the Divide-and-conquer algorithm design technique is that it is **easier to solve several smaller instances of a problem** than the larger one.
- The "divide-and-conquer" technique involves solving a particular problem by dividing it into one or more cub-problems of smaller size, recursively solving each sub-problem and then "merging" the solution of sub-problems to produce a solution to the original problem.

Divide-and-conquer algorithms work according to the following general plan.

- **Divide**: Divide the problem into several smaller sub-problems ideally of about the same size.
- **Conquer**: The smaller sub-problems are solved, typically recursively. If the sub-problem sizes are small enough, just solve the sub-problems in a straightforward manner or conventional methods.
- **Combine**: If necessary, the solution obtained for the smaller problems is merged to get the solution to the original problem.



Algorithm for Divide and Conquer Method: The Control abstraction for divide-and-conquer technique is as follows.

```
Algorithm DandC(p)
{
        if small (p) then
                return Solution(p)
        else
        {
                Divide P into small instances P1, P2, P3……..Pk ;
                Call or Apply DandC to each of these sub-problems ;
                return combine_solutions (DandC(P1), DandC(P1),…. (DandC(Pk) ;
        }
}
```

DandC(p) is the divide-and-conquer algorithm, where P is the problem to be solved. Small(p) is a Boolean valued function (i.e., either true or false) that determines whether the input size is small enough that the answer can be computed without splitting. If this is so the function is invoked. Otherwise, the problem P is divided into smaller sub-problems. These sub-problems P1, P2, P3…….Pk, are solved by receive applications of DandC. Combine is a function that combines the solution of the K sub-problems to get the solution for original problem 'P'.

Recurrence Relation:

The Recurrence Relation is an equation that defines a sequence recursively. The recurrence relations can be solved by the following methods.

1. Substitutional Method
2. Master's Method

1. Substitutional Method: There are two types of substitutional methods

a. Forward Substitution

b. Backward Substitution

a. Forward Substitution: This method makes use of initial condition and value for the next term generated. For example, consider the recurrence relation.

$$T(n) = T(n-1) + n \text{ , with } T(0) = 0$$

If n = 1, T(1) = T(0) + 1 = 0 + 1 = 1

If n = 2, T(2) = T(1) + 2 = 1 + 2 = 3

If n = 3, T(3) = T(2) + 3 = [T(1) + 2]+3 = [T(0) + 1]+2+3 = 0+1+2+3

.
.
.

If n = n, T(n) = 1+2+3+ …+n

By observing the above generated equation, we consider the formula.

$$T(n) = n(n+1) / 2 = \mathbf{O(n^2)}$$

b. Backward Substitution: In this method, backward values are substituted recursively to derive some formula. For example, consider a recurrence relation.

T(n) = T(n-1) + n , with T(0) = 0 → (1)

If n=n-1, T(n-1) = T(n-1-1) + n-1 = T(n-2) + (n-1) → (2)

If n= n-2, T(n-2) = T(n-2-1) + (n-2) → (3)

Substitute Equations (2) & (3) in (1)

T(n) = T(n-1) + n

T(n) = [T(n-2) + (n-1)] + n

T(n) = [T(n-3) + (n-2)] + (n-1) + n

.
.
.

T(n) = T(0) + (n-(n-1)) + … + n

T(n) = 0 + 1 + 2 + 3 + … + n

$$T(n) = n(n+1) / 2 = \mathbf{O(n^2)}$$

2. Master's Method: Consider the following recurrence relation

T(n) = aT(n/b)+f(n), where a ≥ 1, b>1 are positive integers, f(n) be a non-negative function of $O(n^d)$ where d ≥0 in recurrence relation

i. $T(n) = O(n^d)$, *if* $a < b^d$
ii. $T(n) = O(n^d \log n)$ *if* $a = b$
iii. $T(n) = O(n^{\log_b a})$ *if* $a > b^d$

Example: Consider the recurrence relation $T(n) = 8T(n/2) + cn^2$, where c is some positive constant. by applying the master method, and that a=8, b=2, d=2 and $f(n) = cn^2$.

It satisfies $a > b^d$, so this falls into case 3 then $T(n) = O(n^{\log_2 8}) = O(n^3)$, Therefore, T(n) is $\Theta(n^3)$.

Application of Divide and Conquer:

- Binary Search
- Finding the maximum and minimum
- Merge Sort
- Quick Sort
- Defective Chess board

Finding the Maximum and Minimum:

Let P = (n, a[i], …,a[j]) denote an arbitrary instance of the problem. Here n is the number of elements in the list a[i], … , a[j] and we have to find maximum and minimum of this list.

Let small(P) be true when n ≤ 2. In this case, the maximum and minimum are a[i] if n=1. If n=2, the problem can be solved by making one comparison.

Divide: If the list has more than two elements, P has to be divided into smaller instances. For example, P might divide into two instances P_1 = (n/2, a[i], ..., a[n/2]) and P_2 = (n/2, a[n/2+1], …, a[j]). After having divided into two smaller sub problems, we can solve them by recursively invoking the same divide and conquer algorithm.

Conquer: Let Max(P) and Min(P) be the maximum and minimum elements of P, then Max(P) is the larger of $Max(P_1)$ and $Max(P_2)$, Min(P) is the smaller of $Min(P_1)$ and $Min(P_2)$.

Algorithm:

```
Algorithm MaxMin(i, j, max, min)
{
   if (i = j) then max:=min:=a[i]; // Small(P)
   else if (i = j-1) then
      {   if (a[i]<a[j])then
          {
          max :=a[j]; min :=a[i];
          }
          else
          {
          max:=a[i]; min:=a[j]; }
      }
```

```
    else
      {
        mid:=[(i+j)/2];
        MaxMin(i,mid,max,min);
        MaxMin(mid+l,j,maxl,minl);
        if (max<maxl) then max:=maxl;
        if (min >mini)then min:=minl;
      }
}
```

Analysis: Computing time of Finding maximum and minimum element in the array is given the following recurrence relation.

$$T(n) = \begin{cases} T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + 2, & for\ n > 2 \\ 1, & for\ n = 2 \\ 0, & for\ n = 1 \end{cases}$$

From the recurrence relation T(n) = 2T(n/2) + 2 → (1)

if n = n/2 then T(n/2) = 2T(n/4) + 2 → (2)

if n = n/4 then T(n/4) = 2T(n/8) + 2 → (3)

substitute equations (2) & (3) in (1)

T(n) = 2[2T(n/4) +2] + 2 = 4T(n/4) + 4 + 2

T(n) = 4 [2T(n/8) + 2] + 4 + 2

= 8 T(n/8) + 8 + 4 + 2

= 2^3 T(n/2^3) + 2^3 + 2^2 + 2^1

.

= 2^k T(n / 2^k) + 2^k + 2^{k-1} +2^{k-2}+… + 2^1

We know that T(2)=1, so Assume that n / 2^k = 2, n = 2^{k+1}

= 2^k T(2) + 2^k + 2^{k-1} +2^{k-2}+… + 2^1

= 2^k + 2.2^{k-1} + 2.2^{k-2} +2.2^{k-3}+… + 2^1

based on geometric progression

$$\mathbf{a+a.r^1+a.r^2+...+a.r^{n-1}} = \frac{a(r^n-1)}{r-1}$$

a=2, r =2

= 2^k + $\frac{2(2^k-1)}{2-1}$

= 2^k + $\frac{(2^{k+1}-2)}{1}$

= 2^k + 2^{k+1} - 2

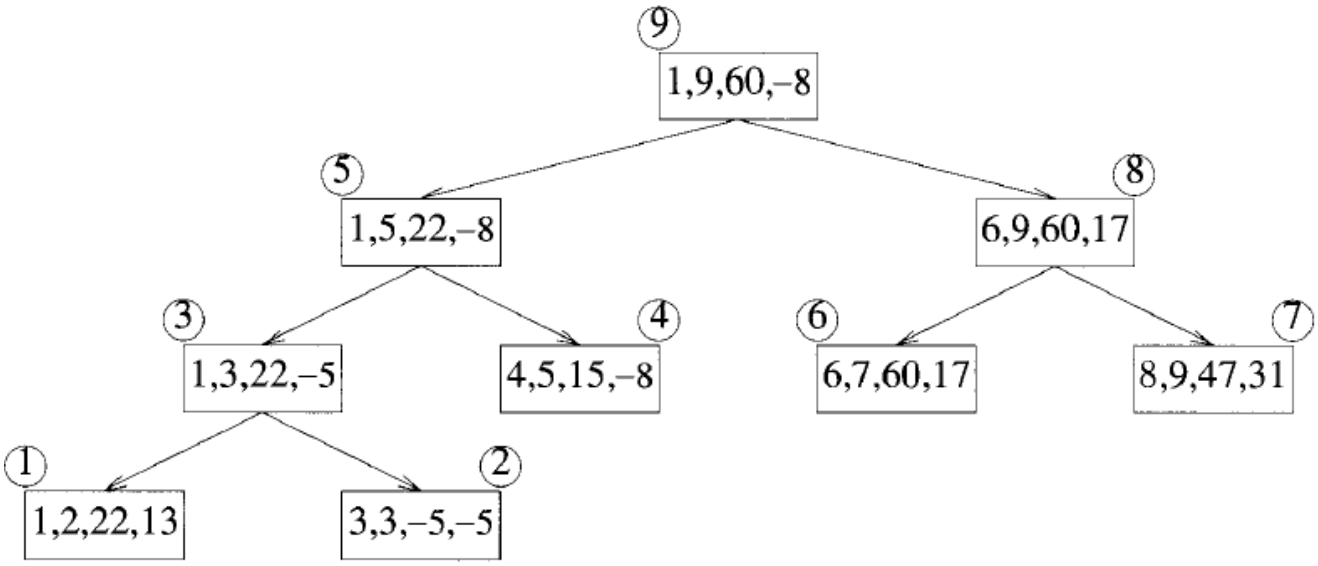
= n/2 + n – 2 [Let n = 2^{k+1}]

= 3n/2 - 2

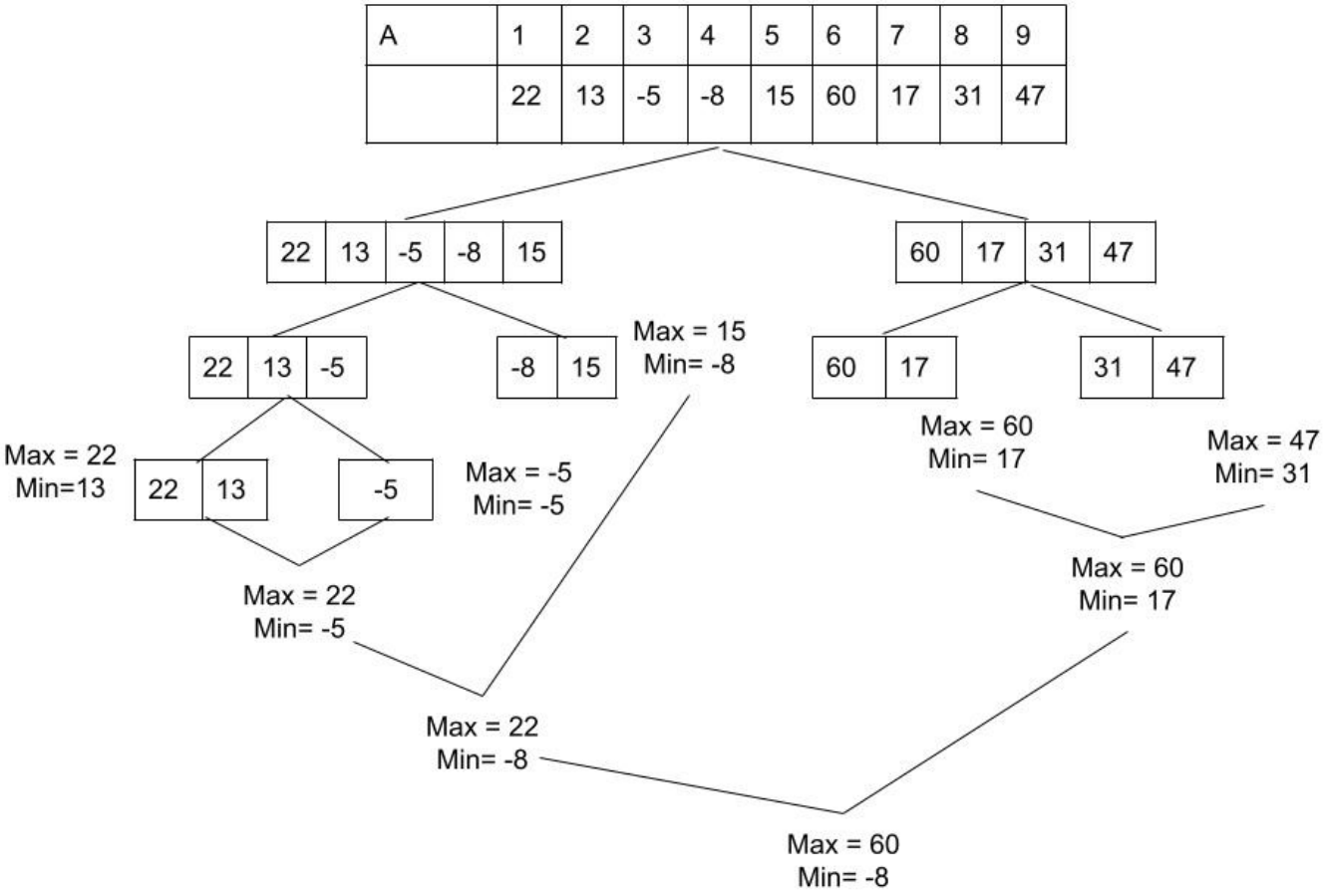
Finally, the time complexity is **T(n) = O(n)**

The procedure is initially invoked by the statement MaxMin(*lb*,*ub*,*x*,*y*), where *lb* is the lower boundary of the list, *ub* is the upper boundary of the list, *x* represents maximum element in the list and *y* represents minimum element in the list.

A	1	2	3	4	5	6	7	8	9
	22	13	-5	-8	15	60	17	31	47

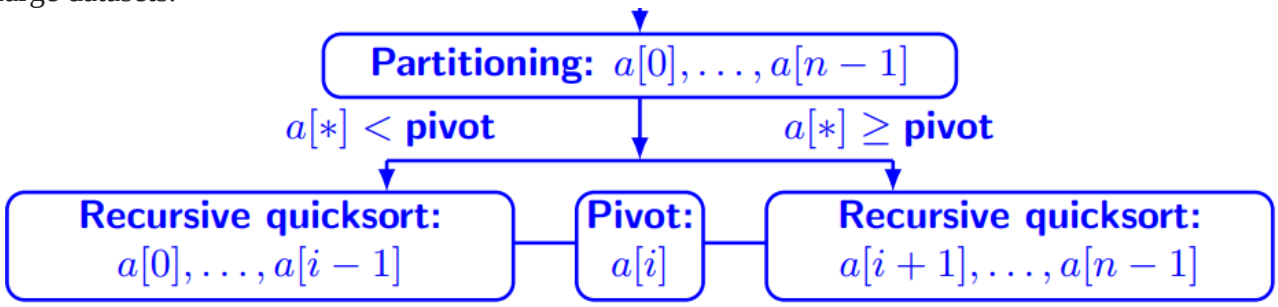


Tree Calls:



Quick sort:

Quick sort follows **Divide and Conquer** algorithm. It is dividing array into smaller parts based on partitioning and performing the sort operations on those divided smaller parts. Hence, it works well for large datasets.



So, here are the steps on how Quick sort works in simple words.

1. First select an element which is to be called as **pivot** element.
2. Next, compare all array elements with the selected pivot element and arrange them in such a way that, elements less than the pivot element are to it's left and greater than pivot is to it's right.
3. Finally, perform the same operations on left and right side elements to the pivot element.

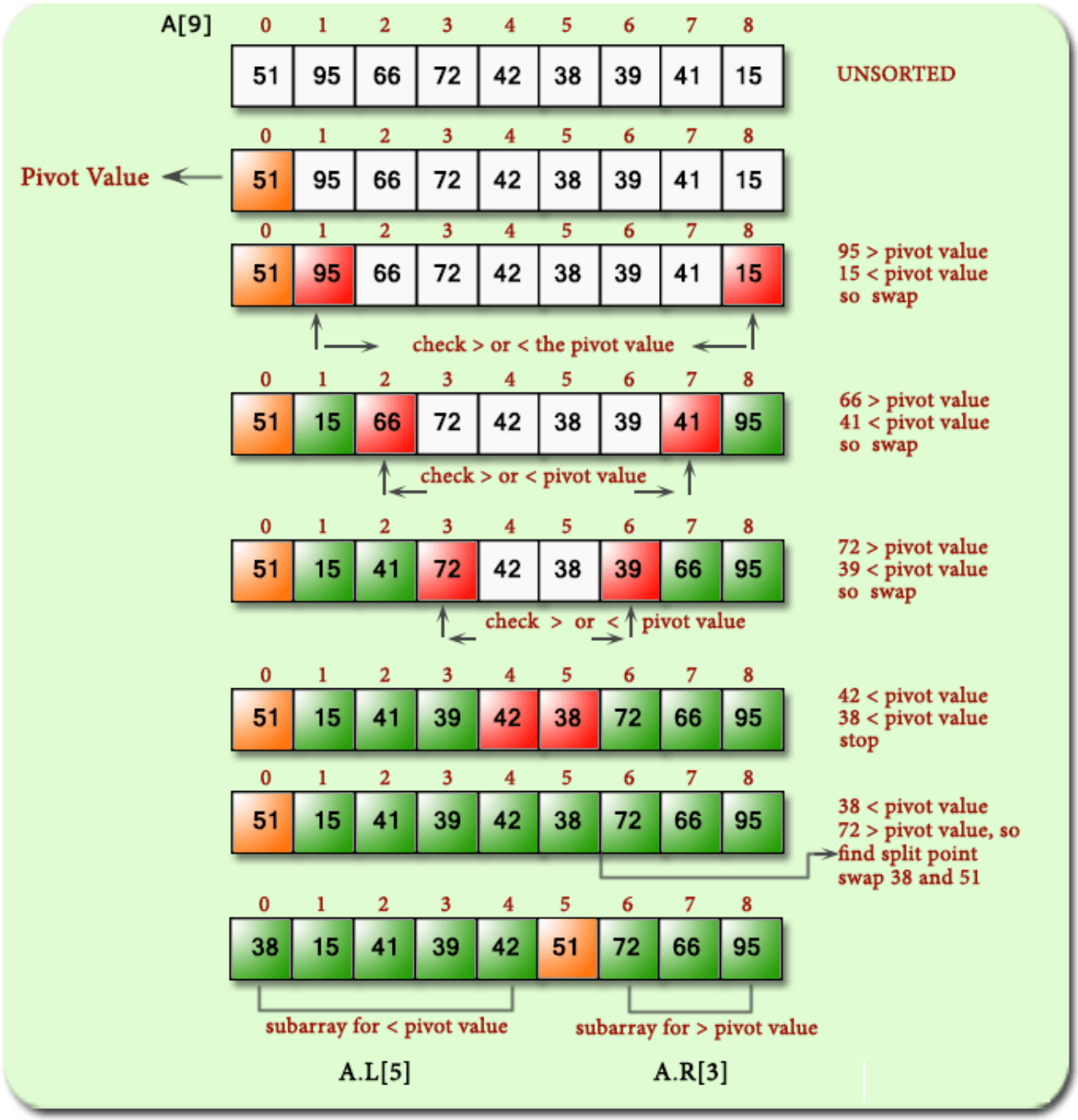
How does QuickSort Partitioning Work

1. First find the "**pivot**" element in the array.
2. Start the left pointer at first element of the array.
3. Start the right pointer at last element of the array.
4. Compare the element pointing with left pointer and if it is less than the pivot element, then move the left pointer to the right (add 1 to the left index). Continue this until left side element is greater than or equal to the pivot element.
5. Compare the element pointing with right pointer and if it is greater than the pivot element, then move the right pointer to the left (subtract 1 to the right index). Continue this until right side element is less than or equal to the pivot element.
6. Check if left pointer is less than or equal to right pointer, then swap the elements in locations of these pointers.
7. Check if index of left pointer is greater than the index of the right pointer, then swap pivot element with right pointer.

Algorithm:

```
Algorithm quickSort(array, lb, ub)
  {    if(lb< ub)
     {       pivotIndex = partition(arr, lb, ub);
             quickSort(arr, lb, pivotIndex - 1);
             quickSort(arr, pivotIndex+1, ub);
     }
  }
```

Example:



Analysis: of Quick Sort

Best and Average Cases: If the array is always partitioned at the mid, then it brings the best-case efficiency of an algorithm. Then the recurrence relation is

T(n) = Time taken to sort left sub array + Time taken to sort right sub array + Time required for partitioning the sub array $T(n) = T(\frac{n}{2}) + T(\frac{n}{2}) + n$

$$T(n) = \begin{cases} 1, & n = 1 \\ 2T(\frac{n}{2}) + n, & n > 1 \end{cases}$$

$T(n) = 2T(\frac{n}{2}) + n \quad \rightarrow (1)$

If $n=n/2$ then $T(\frac{n}{2}) = 2T(\frac{n}{4}) + \frac{n}{2} \quad \rightarrow (2)$

If $n=n/4$ then $T(\frac{n}{4}) = 2T(\frac{n}{8}) + \frac{n}{4} \quad \rightarrow (3)$

Substitute Equations (2) & (3) in (1)

$T(n) = 2\ [2T(\frac{n}{4}) + \frac{n}{2}] + n = 4T(\frac{n}{4}) + n + n = 4T(\frac{n}{4}) + 2n$

$T(n) = 4[2T(\frac{n}{8}) + \frac{n}{4}] + 2n$

$T(n) = 8T(\frac{n}{8}) + n + 2n \quad = 8T(\frac{n}{8}) + 3n = 2^3\ T(\frac{n}{2^3}) + 3n$ *[Let $2^3 = n \rightarrow 3 = \log n$]*

$T(n) = n\ T(\frac{n}{n}) + \log n * n$

$T(n) = n\ T(1) + \log n * n = n + \log n * n$ *[Take the high order]*

The Best and average case time complexity for the Quick Sort is ***O(n logn)***

Worst Cases: The worst case for quick sort occurs when the pivot is a minimum or maximum of all the elements in the list (i.e the input list is either ascending or descending order). Hence, the recurrence relation is

$$T(n) = \begin{cases} 1, & n = 1 \\ T(n-1) + n, & n > 1 \end{cases}$$

$T(n) = T(n-1) + n \rightarrow (1)$

If n= n-1 then $T(n-1) = T(n-1-1) + (n-1) = T(n-2) + (n-1) \rightarrow (2)$

If n= n-2 then $T(n-2) = T(n-2-1) + (n-2) \rightarrow (3)$

Substitute equations (2) & (3) in (1), we get

$T(n) = [T(n-2) + (n-1)] + n = T(n-2) + (n-1) + n$

$T(n) = [T(n-3) + (n-2)] + (n-1) + n$

$= \ldots\ldots$

$= T(n-(n-1)) + n-(n-2) + \ldots + (n-1) + n$

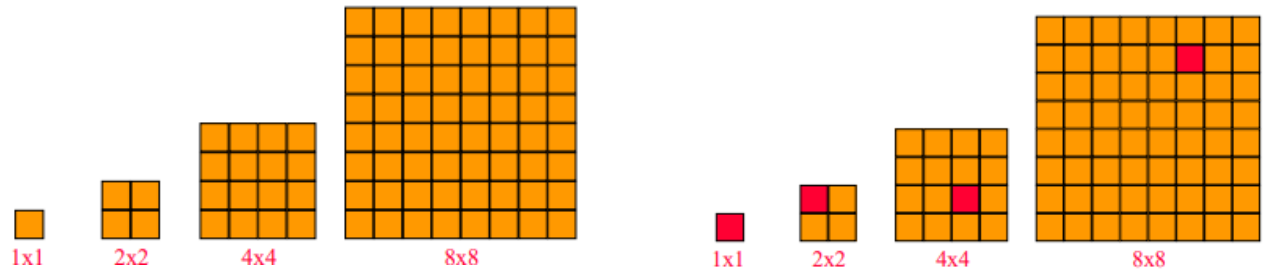
$= 1 + 2 + \ldots + n$

$= \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}$ *[Take the high order]*

The worst-case time complexity for the Quick Sort is ***O(n^2)***

Defective Chess board

- The Defective Chessboard problem, also known as the Tiling Problem.
- It is typically solved with a “divide and conquer” approach.
- A chessboard is an n x n grid, where n is a power of 2.
- A defective chessboard is a chessboard that has one unavailable(defective) position (or) A defective chess board is a $2^k * 2^k$ board of squares with exactly one defective square (where k = 1,2,4,6,8,…..).



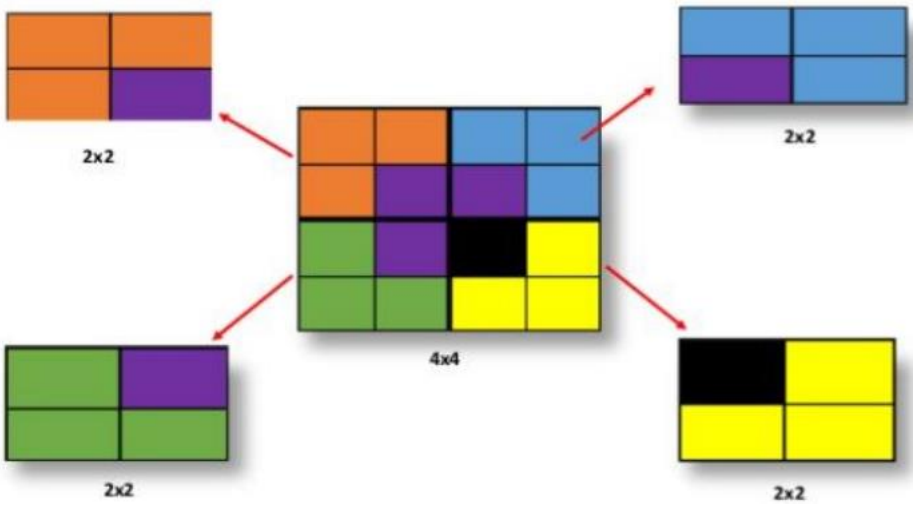
Fill the board using trionimos. A trionimo is an L-shaped tile is a 2 × 2 block with one cell of size 1×1 missing. A triomino has four orientations.



Constraints for Tiling with Triominos

1. Tiling two triominoes may not overlap.
2. Triominoes should not cover defective square.
3. Triominoes should cover all other non-defective squares.

- So, the number of triominoes used becomes $(2^{2k}-1)/3$.
- When k=0 it has no defective square and number of triominoes is 0.
- When k=1, there are exactly three non-defective squares and these squares are covered using a triomino considered as a small(p).
- divide-and-conquer (DAC) technique is used to solve the problem. DAC entails splitting a larger problem into sub-problems, ensuring that each sub-problem is an exact copy of the larger one.
- The method suggests reducing the problem of tiling a $2^k * 2^k$ defective chess board to that of tiling smaller defective chess boards. A natural partitioning of a $2^k * 2^k$ chess board would be in to four $2^{k-1} * 2^{k-1}$ chessboards.
  - Let the 4X4 chessboard can be partitioning when k=2, as follows with 2X2 chess boards (k=1)



To solve this problem using divide and conquer method.

- Base Case: If the board size is 2x2, the defective square is the only square remaining.

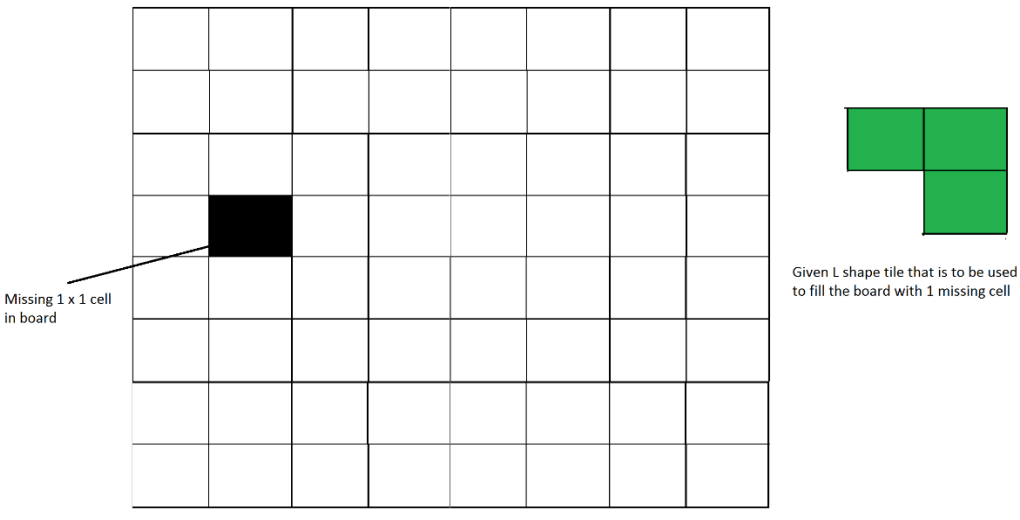
- Divide: Divide the board into four equal-sized subboards.
- Conquer:
    - Place a Triomino: If the defective square is not in any of the subboards, place a triomino in the center to cover the corners of three subboards. The remaining subboard will have the defective square.
    - Recurse: Recursively solve the tiling problem for the subboard with the defective square.

Algorithm:

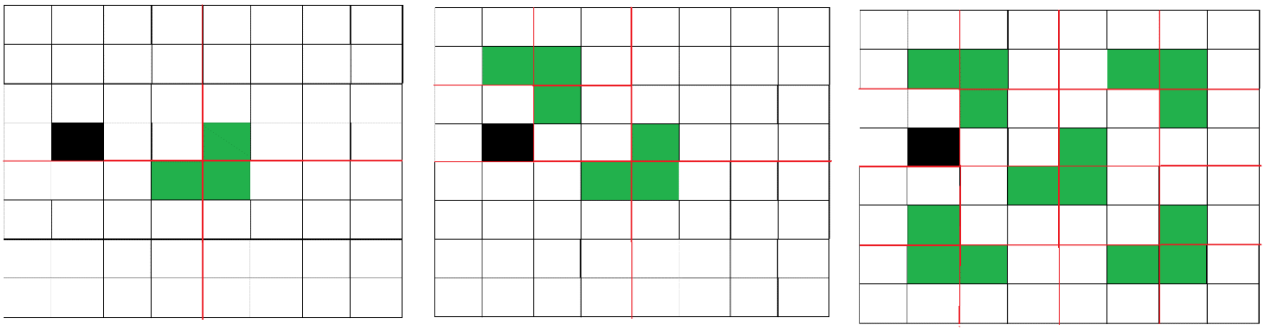
```
Algorithm TileTriminos(n, Tile1)
{
if (k==0) then
    Tile the board with triominos, report failure
else if (k==1) then
    Tile the board with triominos
else if (k>1) then
    Divide the board recursively into 2X2 squares
    Place the triominos at the center, to cover the 3 squares
    Let the l1,l2,l3, and l4 are the divided boards, then
recursively call
        TileTriminos(n-1, l1);
        TileTriminos(n-1, l2);
        TileTriminos(n-1, l3);
        TileTriminos(n-1, l4);
}
```

Example:

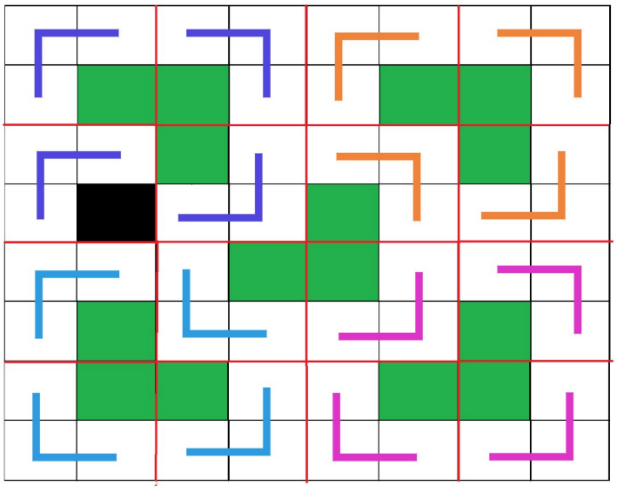
Given a n X n board where n is of form 2^k where k >= 1 (Basically n is a power of 2 with minimum value as 2). The board has one missing cell (of size 1 X 1). Fill the board using L shaped tiles. A L-shaped tile is a 2 X 2 square with one cell of size 1X1 missing (Triomino).



The below diagrams show working of algorithm, tile the non-defective squares with triomino at corners of the three squares, later divide the squares into sub-squares and tile the non-defective squares with triominos, this procedure will be repeated for all the sub-squares until the problem is small. When the problem is small then the squares directly tile with Triominos.



After tiling the all the sub-squares of 2X2 with triominos as follows



Analysis:

The recurrence relation for the tile the defective chessboard is

$$T(n) = \begin{cases} c, & k = 1 \\ 4T(n-1) + c, & k > 1 \end{cases}$$

T(n) = 4T(n-1) + c $\rightarrow$ *(1)*

If n= n-1 then T(n-1) = 4T (n-1-1) + c = 4T(n-2) + c $\rightarrow$ *(2)*

If n= n-2 then T(n-2) = 4T (n-2-1) + c = 4T(n-3) + c $\rightarrow$ *(3)*

Substitute equations (2) & (3) in (1), we get

$T(n) = 4[4T(n-2) + c\ ] + c = 4^2T(n-2) + 4c + c$

$= 4^2[4T(n-3) + c] + 4c + c = 4^3T(n-3) + 4^2c + 4c + c$

...

$= 4^kT(1) + 4^{k-1}c + \ldots + c = 4^kc + 4^{k-1}c + \ldots + c$

$= c\ [\ 4^k + 4^{k-1} + \ldots + 1]$

Applying the Geometric Series Formula: $4^0 + 4 + \ldots + 4^{k-1} + 4^k$

The first term is a=1, The common ratio is r=4, There are k+1 terms.

$$s = \frac{a(r^n - 1)}{r - 1} = \frac{1(4^{k+1} - 1)}{4 - 1} = \frac{4^{k+1} - 1}{3}$$

The time complexity for the defective chess board is ***O(4^k)*** *is proportional to* ***O(n^2)***